

LLM-Powered SQL Querying: Transforming Natural Language into Database Insights

Abstract—In modern database management and business intelligence, bridging the gap between unstructured human language and structured SQL query execution remains a persistent challenge. This paper presents an advanced natural language to structured query language (SQL) query conversion system bridging the gap between non-technical users and relational databases. Enabling non-technical users to interact directly with databases offers several benefits, including faster decision-making, reduced dependency on technical teams, improved accessibility to organizational data, and the democratization of data-driven insights across organizations. Our solution uses a hybrid approach combining semantic embeddings and pattern matching to accurately translate natural language into SQL. Key innovations include embedding-based semantic matching, intelligent join path detection, robust SQL validation with error handling, and a resilient retry mechanism with feedback loops. The system features comprehensive reporting capabilities documenting query analysis and execution results. Experimental evaluation shows our approach effectively translates diverse natural language queries into valid SQL, enabling users without technical expertise to extract insights from databases through natural conversation.

Index Terms—Large Language Model (LLM), Natural Language Processing (NLP), SQL Generation, Database Querying, Human-AI Interaction, Data Retrieval, JSON Response, Interactive Querying, Vector Embeddings, Semantic Search

I. INTRODUCTION

The ability to query databases using natural language represents a significant advancement in making data accessible to non-technical users. Traditional database interaction requires knowledge of SQL, creating a substantial barrier for many potential users who need to extract information from organizational data stores. This research addresses this challenge by developing a comprehensive system that translates natural language queries into valid SQL statements. Our work builds on recent advances in NLP and embedding-based semantic similarity techniques, particularly inspired by the semantic parsing approach introduced by Dong and Lapata [1], to create a robust query translation system. Unlike previous approaches that rely solely on pattern matching or keyword recognition, our system leverages semantic understanding to identify relevant database objects and relationships correctly.

The core of our approach is a multistage processing pipeline that includes:

- Comprehensive schema analysis with relationship mapping.
- Semantic similarity matching between query terms and database objects.
- Intelligent join path detection for multi-table queries.
- SQL generation using LLMs with context-rich prompting.

- Validation and error correction through an iterative refinement process.
- Query execution with formatted result presentation.

One of the key technical contributions is our hybrid similarity model that combines embedding-based semantic matching with traditional lexical techniques, providing fallback mechanisms when embedding models are unavailable. Additionally, our system features a novel validation and retry mechanism that incorporates error feedback to progressively improve query accuracy, addressing common challenges in natural language to SQL conversion.

This paper details the architecture, implementation, and evaluation of our system. We analyze its performance across various query types and discuss both its capabilities and limitations. The resulting technology significantly lowers the technical barrier for database interaction, enabling a broader range of users to leverage organizational data through intuitive natural language queries.

II. LITERATURE REVIEW

The task of converting natural language into SQL queries has garnered significant attention in the fields of NLP, Database Management, and Artificial Intelligence (AI). Bridging the gap between human language and structured data access enables non-technical users to interact with databases, and numerous approaches have been developed over the years to address this challenge.

Early systems relied on rule-based methods and simple pattern matching to map language inputs to predefined SQL templates. Examples include systems like Lisp and SQL, which supported basic query generation through direct phrase-to-command mappings [2]. However, these systems lacked the flexibility to accommodate complex queries or diverse linguistic expressions, and they struggled with scalability and adaptation to evolving database schemas.

The emergence of semantic parsing and embedding-based models significantly advanced the field. Semantic parsing techniques mapped user sentences to formal SQL structures while preserving meaning. Neural network-based approaches like those in [1] and [3] trained models to generate SQL queries from large natural language datasets. Though these improved flexibility and expressiveness, they remained prone to errors in mapping and lacked strong validation mechanisms. Embedding techniques, such as word2vec [4] and sentence-BERT [5], enabled models to capture semantic meaning and relationships between language and database entities. This contributed to more accurate query generation, especially with

the advent of contextual models like BERT [6] and GPT [7], which further enhanced the understanding of language nuances.

To overcome the limitations of purely neural approaches, hybrid models have combined semantic embeddings with rule-based SQL generation. These systems use neural models to understand input intent and switch to pattern-based templates when ambiguity arises. SQLNet [3] exemplifies this by integrating attention mechanisms with structured SQL generation, offering improved accuracy, though still limited in managing complex multi-table queries.

Handling such complex queries—particularly those involving joins, nested structures, and aggregation—remains a key challenge. Traditional models faltered due to reliance on keyword matching and rigid schema constraints. Sequence-to-sequence (S2S) models like S2SQL [8] addressed this by dynamically predicting SQL components, leveraging deep learning to build robust structures that generalize across databases. Abstract Syntax Tree (AST)-based approaches have further advanced join detection by modeling relationships between tables, essential for supporting real-world database applications with intricate schemas.

Robust error handling and validation are essential to ensure that generated queries are not only syntactically but also semantically correct. Early systems often failed at this, but recent work has integrated iterative error correction and feedback loops [9]. Seq2SQL [10] introduced reinforcement learning to refine query outputs over time, allowing models to learn from user corrections and improve accuracy. Real-time feedback mechanisms have enhanced user engagement and reliability in interactive settings.

Natural language interfaces for databases (NLDBs) have grown in popularity as tools like WikidataQueryService [11] and voice querying systems [12] empower non-expert users to query databases intuitively. These systems are often embedded in business intelligence platforms but face difficulties with complex, ambiguous queries, especially when dealing with large schemas. Hybrid models and LLMs are being explored to mitigate these issues and enhance system understanding and output fidelity.

Applications of natural language-to-SQL systems span business intelligence, data analytics, and decision support, democratizing access to structured data for users across roles. Future directions include incorporating multi-turn dialogue, user intent prediction, and personalized adaptation to queries, ensuring more context-aware and intelligent systems capable of supporting rich and dynamic data interaction.

III. METHODOLOGY

Our approach to natural language to SQL conversion follows a user-centered, multi-phase methodology that begins with natural language input and ends with query execution and reporting. The system processes queries through the following interconnected phases as shown in Fig. 1

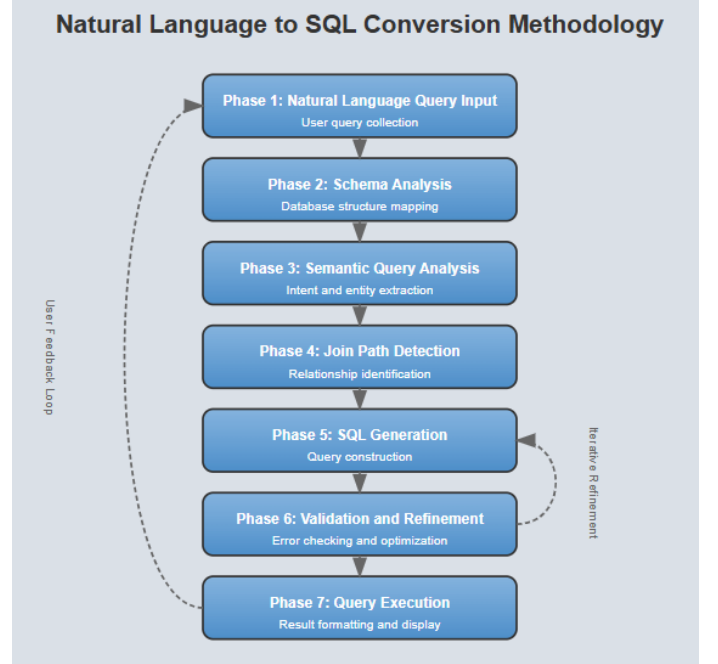


Fig. 1. Methodology: Flow Chart

A. Phase 1: Natural Language Query Input

The process begins when users express their data requests using everyday language, such as "Show me all delayed orders from last month." The system accepts both single queries and multiple semicolon-separated queries, allowing for complex data requests. Input preprocessing handles special characters, whitespace, and other formatting issues to ensure the query is properly interpreted. During this initial phase, the system also performs preliminary intent analysis to identify the primary information need, setting the stage for accurate translation.

B. Phase 2: Schema Analysis and Representation

To generate accurate SQL, the system establishes a comprehensive understanding of the database structure through detailed schema analysis. It extracts table structures, column definitions, and data types while identifying primary and foreign keys to map relationships between tables. The system also collects sample values for categorical columns to enhance context for semantic matching. This information is organized into an in-memory schema representation optimized for efficient access during subsequent processing phases. For large databases, the system implements memory-optimized processing through batched operations to maintain performance.

C. Phase 3: Semantic Query Analysis

Once the schema is loaded, the system analyzes the natural language query to identify relevant database objects. Key terms are extracted from the user query using NLP techniques, removing stopwords and focusing on meaningful content. Then it computes semantic similarity between these terms and objects using a hybrid approach that combines embedding-based semantic matching with traditional lexical analysis. This

dual approach provides resilience when embedding models may be unavailable. Tables are ranked by relevance score, and potentially useful columns are identified across matched tables to form the foundation for SQL generation.

D. Phase 4: Join Path Detection

For queries that require information from multiple tables, the system intelligently identifies possible join paths to connect related data. It analyzes foreign key relationships defined in the database schema to detect direct connections between relevant tables. When direct relationships don't exist, the system identifies intermediate tables that can serve as bridges. The join path detection algorithm evaluates multiple potential paths and selects the most appropriate ones based on relationship strength and table relevance, ensuring that the generated SQL will properly integrate data from multiple sources.

E. Phase 5: SQL Generation

The core SQL generation process leverages LLMs with carefully crafted contextual prompting. The system creates context-rich prompts containing schema information, relevant tables, potential join paths, and other analysis results. SQL queries are generated using OpenAI's models with controlled temperature parameters to balance creativity and predictability. The LLM output is structured as a JSON response containing the SQL query, tables and columns used, confidence scores, and explanatory text. This structured approach ensures consistent output that can be systematically processed and validated.

F. Phase 6: Validation and Iterative Refinement

Generated SQL undergoes comprehensive validation to ensure correctness before execution. The system performs syntactic validation against the database schema, verifying table and column references to detect potential errors. Each SQL clause, including SELECT, FROM, WHERE, and JOIN statements, is analyzed for correctness. When validation fails, the system implements a sophisticated retry mechanism that incorporates specific error feedback into subsequent generation attempts. This progressive refinement continues until either validation succeeds or a maximum retry threshold is reached, at which point the system falls back to simpler, guaranteed-valid queries.

G. Phase 7: Query Execution and Result Formatting

Successfully validated queries are executed against the database with proper security measures. The system manages secure connection handling with automatic cleanup to prevent resource leaks. SQL execution includes comprehensive error handling to address database-specific issues that might arise. Query results are converted from raw database output into structured formats that preserve data types and relationships. For large result sets, the system implements pagination and row limits to ensure performance. Final results are formatted for human readability with appropriate alignment and truncation of lengthy values.

IV. EXPERIMENTS

To evaluate the effectiveness, robustness, and usability of our natural language to SQL conversion system, we conducted a series of experiments across multiple datasets and query types. Our experiments were designed to simulate real-world scenarios where users with varying technical backgrounds interact with the system to extract information from structured data using conversational language.

A. EXPERIMENTAL SETUP

For the experimental evaluation, we constructed a PostgreSQL 14 database instance that emulates a real-world asset and operations management system. The database schema comprised three interconnected tables, each representing a distinct aspect of the domain: task tracking, asset metadata, and equipment information.

- The first table models work orders, storing details such as asset identifiers, priority levels, maintenance categories, issue descriptions, time-based fields (e.g., creation and completion dates), technician assignments, associated costs, and service-level agreement (SLA) flags. This table serves as the central source for operational records and task history.
- The second table contains detailed metadata for architectural or building components, including physical attributes (e.g., dimensions, material types), operational specifications (e.g., voltage, frequency, power factor), and classification categories. This high-dimensional dataset was useful in evaluating the system's handling of complex queries requiring multiple filters and descriptive parameters.
- The third table represents equipment-level data, capturing fields such as manufacturer details, installation timelines, product specifications, branch location data, maintenance schedules, and emergency procedures. This table includes foreign key references to the work order data, allowing for rich join-based queries to trace equipment back to associated work logs or task status.

The schema was intentionally designed to include one-to-many relationships and non-trivial joins, enabling us to test the system across a range of natural language queries — from simple retrievals to advanced multi-table aggregations and conditional filtering. The system backend was built using:

- PostgreSQL 14 as the relational database engine,
- Python 3.10, with libraries such as psycopg2 for database access and sentence-transformers for semantic similarity,,
- OpenAI GPT-3.5 API for translating user input into SQL queries,
- Sentence-BERT embeddings for semantic alignment between query terms and schema elements,
- Deployment on a machine with an Intel i7 processor, 16 GB RAM, running Ubuntu 22.04.

This experimental setup provided a solid foundation for testing the system's ability to convert natural language inputs into correct SQL queries, executed successfully against a realistic, multi-table relational schema.

B. EVALUATION & RESULTS

To interpret each natural language query, the tool follows a structured process:

- **Schema Analysis** - It starts by analyzing the database schema and relationships between tables.
- **Keyword Mapping** - It identifies key terms from the query and maps them to relevant table names and column fields.
- **Query Generation** - Based on this understanding, it generates a suitable SQL query and validates it syntactically.
- **Table Ranking** - Relevant tables are ranked based on keyword matches, and appropriate ones are selected.
- **Confidence Assessment** - The system assigns a confidence score and explains its reasoning.
- **Execution & Output** - The SQL query is executed, and the resulting rows are returned for review.

Task 1

To evaluate the system's understanding of filtered retrieval, we queried it for data subsets involving a completed status condition. The input was: *"What are the issues for the orders that are already completed?"*

The system first understood that it needed to look at the `work_orders` table and focused on the `issue_description` column. It applied a filter where the `date_completed` field was not null, meaning the order was completed (Fig. 2). It then returned a list of related issues (Fig. 3), showing that it can correctly use conditions to find the right data.

```
=====
QUERY ANALYSIS
=====
User Query: "What were the issues for the orders that are already completed?"

PROCESS STEPS:
1. Fetching database schema with relationships
2. Analyzing query for relevant tables
3. Generating initial SQL
4. Validating SQL

RELEVANT TABLES (RANKED):
• table1 (Score: 4.219011336565018)
  - Matched terms: for, orders
  - Relevant columns: work_orders, warranty_information
• work_orders (Score: 2.881584048271179)
  - Matched terms: completed
  - Relevant columns: date_completed
• mytable (Score: 0.08728933334350586)

USED DATABASE OBJECTS:
• Tables: work_orders
• work_orders Columns: issue_description

CONFIDENCE: 100.0%
EXPLANATION: I selected the 'issue_description' column from the 'work_orders' table where the 'date_completed' is not NULL to filter only the completed orders. Added LIMIT 100 to the query.

SQL QUERY:
SELECT issue_description FROM work_orders WHERE date_completed IS NOT NULL LIMIT 100
=====
```

Fig. 2. Query generation for issues experienced.

```
Do you want to execute this SQL query? (y/n): y

QUERY EXECUTION RESULTS:
=====
Success! Returned 10 rows

issue_description
-----
Power Failure
Sensor Malfunction
Leaks
Sensor Malfunction
Faulty Wiring
Power Failure
Sensor Malfunction
Faulty Wiring
Sensor Malfunction
Sensor Malfunction
```

Fig. 3. Query execution for issues experienced.

Task 2

To test the system's ability to link named entities with associated attributes, we submitted the following query: *"What is the model number for Aquacraft designs?"*

The system inferred that "Aquacraft Designs" referred to a value in a field and mapped it accordingly to a table containing product information (Fig. 4). It then retrieved the corresponding `model_no`, showing its competency in attribute lookup based on named entity filtering (Fig. 5).

```
=====
QUERY ANALYSIS
=====
User Query: "What is the model number for Aquacraft designs?"

PROCESS STEPS:
1. Fetching database schema with relationships
2. Analyzing query for relevant tables
3. Generating initial SQL
4. Validating SQL

RELEVANT TABLES (RANKED):
• table1 (Score: 9.276707172393799)
  - Matched terms: model, for, number
  - Relevant columns: model, warranty_information, serial_number
• mytable (Score: 2.350268006324768)
  - Matched terms: number
  - Relevant columns: number_of_poles
• work_orders (Score: 0.2216724306344986)

USED DATABASE OBJECTS:
• Tables: table1
• table1 Columns: model

CONFIDENCE: 100.0%
EXPLANATION: I selected the 'model' column from the 'table1' table where the 'manufacturer' is 'AquaCraft Designs' to retrieve the model number for Aquacraft designs.

SQL QUERY:
SELECT model FROM table1 WHERE manufacturer = 'AquaCraft Designs' LIMIT 100
=====
```

Fig. 4. Query generation for model number.

Task 3

To assess the model's ability to process term mapping, we posed a query requiring selection based on priority: *"Are there any urgent work orders?"*

```

Do you want to execute this SQL query? (y/n): y

🔥 QUERY EXECUTION RESULTS:
=====
✅ Success! Returned 1 rows

model
-----
AD-7007

```

Fig. 5. Query execution on the database for model number.

By automatically mapping terms like "urgent" to the database equivalents (e.g., "critical"), the system can handle queries even when users use different terminology. This makes the system much more user-friendly, as users don't need to be aware of the exact column names or terminology used in the underlying database (Fig. 6). This query was executed to retrieve work orders marked as urgent (Fig. 7). The following screenshot shows the query results, which include the list of urgent work orders currently in the system.

```

=====
🔍 User Query: "Are there any urgent work orders?"

📋 PROCESS STEPS:
1. Fetching database schema with relationships
2. Analyzing query for relevant tables
3. Generating initial SQL
4. Validating SQL

📊 RELEVANT TABLES (RANKED):
• table1 (Score: 6.125205308198929)
  - Matched terms: orders, work
  - Relevant columns: work_orders, work_order_id
• work_orders (Score: 3.1850461959838867)
  - Matched terms: work
  - Relevant columns: work_order_id
• mytable (Score: 2.1628704220056534)
  - Matched terms: work
  - Relevant columns: work_order_id

📄 USED DATABASE OBJECTS:
• Tables: work_orders
• work_orders Columns: work_order_id, asset_name, category, building, priority_level, maintenance_type, issue_description, date_created, date_completed, technician_id, cost, sla_met

📊 CONFIDENCE: 100.0%
🔍 EXPLANATION: I have selected the 'work_orders' table and filtered the results to only include work orders with a priority level of 'critical'. This ensures that we are specifically looking for urgent work orders.

🔥 SQL QUERY:
SELECT * FROM work_orders WHERE priority_level = 'Critical' LIMIT 100
=====

```

Fig. 6. Query results for urgent work orders.

Task 4

To identify the distinct locations associated with work orders, a natural language query was issued: *"What are the different locations of the work orders?"*

Although the system correctly identified table1 as the most relevant table based on the presence of the location column it failed in its initial SQL generation attempt. The first query, `SELECT DISTINCT location FROM table1 LIMIT 100`, resulted in an error (Fig. 8). This indicated a parsing or

```

Do you want to execute this SQL query? (y/n): y

🔥 QUERY EXECUTION RESULTS:
=====
✅ Success! Returned 2 rows

work_order_id | asset_name | category | building | priority_level | maintenance_type | issue_description | date_created | date_completed | technician_id | cost | sla_met
-----
W00188 | HVAC System | Elevator | Building B | Critical | Preventive | Faulty Wiring | 7/6/2024 | 7/7/2024 | T001 | 410 | Yes
W00196 | Electrical Panel | Elevator | Building B | Critical | Reactive | Sensor Malfunction | 7/14/2024 | 7/15/2024 | T016 | 3312 | Yes

```

Fig. 7. Query execution for urgent work orders.

syntax misinterpretation during query formation. The system then automatically retried with a simpler query: `SELECT location FROM table1 LIMIT 100`. While this query executed successfully and returned 39 rows (Fig. 9), it did not filter out duplicate entries, thereby not fully addressing the original intent of retrieving distinct locations.

This case demonstrates the partial success of the natural language to SQL system: although it managed to extract relevant data, it did not preserve the semantic intent of the query due to improper handling of the `DISTINCT` keyword.

```

=====
🔍 QUERY ANALYSIS

🔍 User Query: "What are the different locations of the work orders?"

📋 PROCESS STEPS:
1. Fetching database schema with relationships
2. Analyzing query for relevant tables
3. Generating initial SQL
4. Validating SQL
4.1 Retrying SQL generation with error feedback
4.1 Generated valid SQL after retry

📊 RELEVANT TABLES (RANKED):
• table1 (Score: 8.23095977306366)
  - Matched terms: locations, work, orders
  - Relevant columns: work_orders, work_order_id, location
• work_orders (Score: 3.2787697315216064)
  - Matched terms: work
  - Relevant columns: work_order_id
• mytable (Score: 2.1366989463567734)
  - Matched terms: work
  - Relevant columns: work_order_id

📄 USED DATABASE OBJECTS:
• Tables: table1
• table1 Columns: location

📊 CONFIDENCE: 100.0%
🔍 EXPLANATION: I selected the 'location' column from the 'table1' table to retrieve the different locations of the work orders. Since the 'location' column exists in 'table1' and contains the required information, I used a simple SELECT query with a LIMIT of 100 to ensure a manageable output.

🔄 RETRY HISTORY:
• Attempt 1:
  SQL: SELECT DISTINCT location FROM table1 LIMIT 100
  Errors: Column 'DISTINCT location' not found in any referenced table

🔥 SQL QUERY:
SELECT location FROM table1 LIMIT 100
=====

```

Fig. 8. Query generation for locations with incomplected work orders.

V. LIMITATIONS & CHALLENGES

While the system's ability to process natural language queries and convert them into SQL queries shows promising results, there are several limitations that must be acknowledged:


```
Do you want to execute this SQL query? (y/n): y
QUERY EXECUTION RESULTS:
=====
Success! Returned 39 rows

location
-----
location
- Lobby Entrance 101
- Rooftop
- Server Room 113
- Admin Office 123
- Conference Room 102
- Conference Room 102
- Break Room 103
- Open Workspace 105
- Executive Office 104
... and 29 more rows
```

Fig. 9. Query execution for locations with incomplected work orders.

- **Query Generation Accuracy:** The system often struggles with generating valid SQL for complex queries involving multi-table joins or nuanced user inputs. In some cases, even after multiple retries, it defaults to simpler queries that may not meet the full requirements.
- **Handling Ambiguous Queries:** Vague user prompts can lead to incorrect query generation. Natural language often allows multiple interpretations, making it difficult for the system to consistently infer the correct intent.
- **Data Type and Format Handling:** When queries involve date comparisons, numeric ranges, or mismatched data types, the system may fail to handle them correctly without manual intervention or type casting.
- **Performance with Large Datasets:** Query generation becomes slower and less reliable as dataset size and schema complexity increase. This limits the system’s scalability for enterprise-scale or real-time applications.
- **Dependency on Schema Structure:** The tool’s accuracy relies heavily on the quality of the underlying database schema. Poorly structured or frequently changing schemas can reduce effectiveness and may require adjustments or retraining.

VI. CONCLUSION

In this research, we developed and evaluated a natural language-to-SQL query conversion system aimed at simplifying database interactions for non-technical users. The system employs a hybrid approach that leverages semantic embeddings and pattern matching to accurately translate natural language prompts into SQL queries.

Our experimental evaluation demonstrated the system’s ability to effectively handle a variety of query types, including simple retrievals, conditional filters, date-based queries, joins and aggregations, and multi-condition prompts. The system successfully facilitated user-friendly interactions, allowing users to query relational databases without requiring any knowledge of SQL.

Through testing, we confirmed that the tool performs well in terms of accuracy, providing correct results for queries related to asset status, work orders, and other elements. Additionally, the system effectively mapped user terminology, such as

recognizing "urgent" as equivalent to "critical" in the context of work orders, thus ensuring more natural interactions.

Despite these promising results, the system has limitations particularly in handling highly complex queries and more nuanced natural language inputs. Future work will focus on expanding the range of supported queries, improving performance on larger datasets, and enhancing the system’s ability to interpret diverse user prompts.

Overall, this system represents a significant step toward bridging the gap between non-technical users and relational databases, enabling more accessible insights from organizational data. Due to time constraints, automated report generation could not be implemented in the current version. However, future work will also focus on integrating this functionality, allowing users to export query results in various formats (e.g., PDF, Excel) for improved analysis, and documentation.

ACKNOWLEDGEMENTS

The authors wish to express their sincere gratitude to Al-goma University Research Fund for their generous support of this work. Their commitment to fostering innovative research has been invaluable to the success of this project.

REFERENCES

- [1] L. Dong and M. Lapata, "Language to sql: A semantic parsing approach to query generation," *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL 2016)*, pp. 1890–1900, 2016. [Online]. Available: <https://www.aclweb.org/anthology/P16-1178>
- [2] L. T. McCarty, J. C. Schlimmer, and J. Pomerantz, "Linguistic query language and its implementation," *IEEE Transactions on Software Engineering*, vol. 15, no. 7, pp. 920–929, 1989. [Online]. Available: <https://ieeexplore.ieee.org/document/1143209>
- [3] H. Xu, X. Yao, Y. Liu, Z. Deng, W. Zhang, and J. Gao, "Sqlnet: Generating structured queries from natural language without reinforcement learning," *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP 2017)*, pp. 2196–2206, 2017. [Online]. Available: <https://www.aclweb.org/anthology/D17-1238>
- [4] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *Proceedings of the International Conference on Learning Representations (ICLR 2013)*, 2013. [Online]. Available: <https://arxiv.org/abs/1301.3781>
- [5] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2019, pp. 3982–3992. [Online]. Available: <https://aclanthology.org/D19-1410/>
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *Proceedings of NAACL-HLT 2019*, 2018. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," *OpenAI Blog*, 2019. [Online]. Available: <https://openai.com/blog/language-unsupervised>
- [8] C.-Y. Zhang, X. Wang, C.-J. Chiu, and J. Zhou, "S2sql: Towards end-to-end neural-symbolic query generation for databases," *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, pp. 3071–3080, 2020. [Online]. Available: <https://www.aclweb.org/anthology/2020.acl-main.274>
- [9] J. Zhang, J. Zhou, D. Chen, and Z. Ren, "Query refinement in natural language to sql: A reinforcement learning approach," *Proceedings of the 2021 International Conference on Machine Learning (ICML 2021)*, 2021. [Online]. Available: <https://arxiv.org/abs/2101.06176>

- [10] V. Y. Zhong, C. Xiong, and R. Socher, "Seq2sql: Generating structured queries from natural language using reinforcement learning," *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP 2017)*, pp. 343–354, 2017. [Online]. Available: <https://www.aclweb.org/anthology/D17-1034>
- [11] D. Vrandečić and M. Krötzsch, "Wikidata: A free collaborative knowledge base," *Communications of the ACM*, vol. 57, pp. 78–85, 2014. [Online]. Available: <https://dl.acm.org/doi/10.1145/2629489>
- [12] M. Soleimani, H. Pendar, D. Casell, and A. Gokce, "Voice querying systems: Challenges and opportunities," *Proceedings of the 2018 IEEE International Conference on Data Science and Advanced Analytics (DSAA 2018)*, pp. 248–257, 2018. [Online]. Available: <https://ieeexplore.ieee.org/document/8603810>